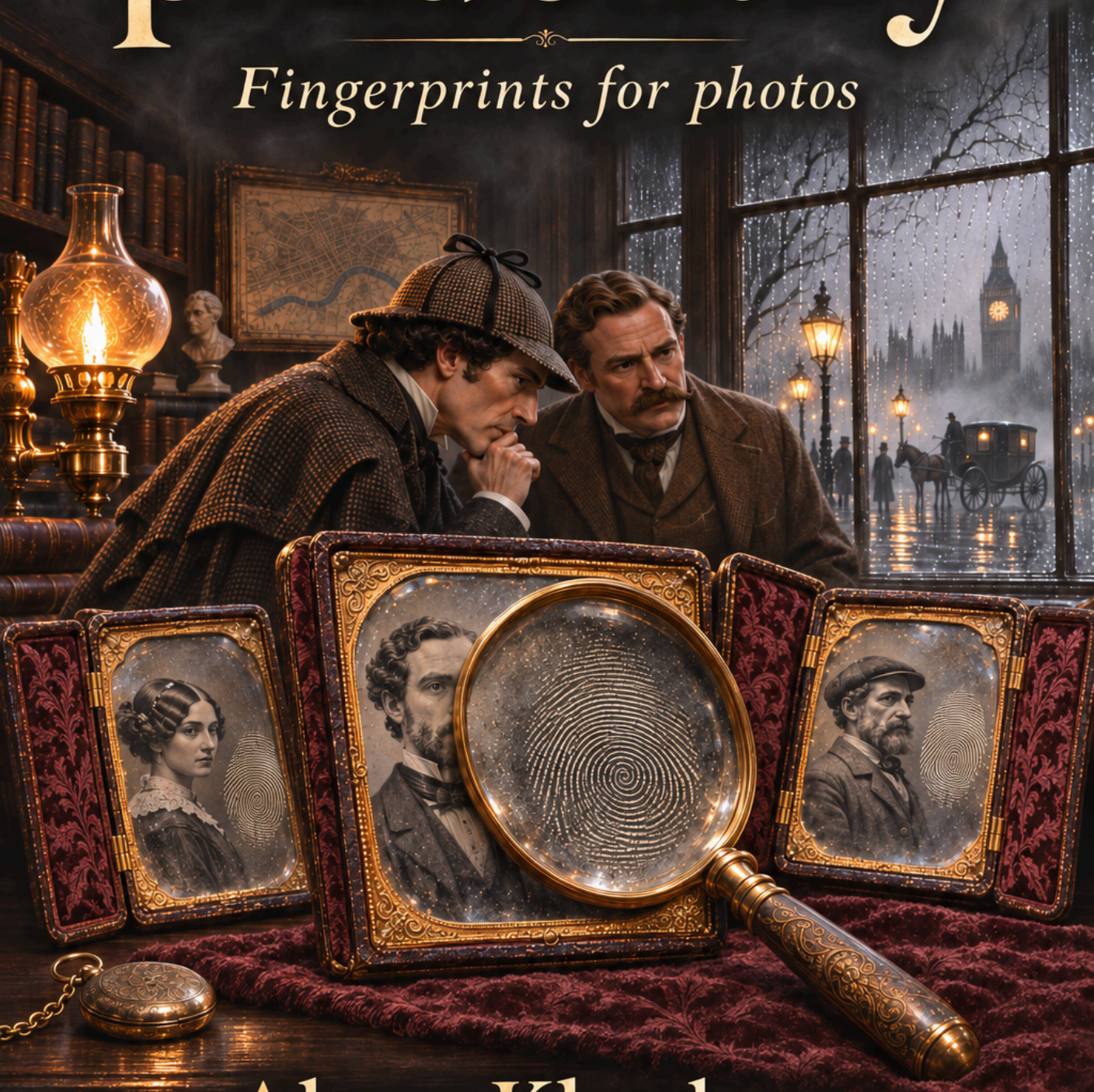
phidentify

Fingerprints for photos



Alexy Khrabrov

FIRST PAIR
PRESS

phidentify

Fingerprints for photos

Alexy Khrabrov

2026-11-10

Version 0.3.0-2fd2e1b0
Source commit 2fd2e1b0341cecb26a4700f902737933627c7bda

First Pair Press
firstpair.press

Contents

About this manual 4
What phidentify is 5
  Where it fits 5
  What it is not 6
Architecture 8
Building 9
  Rust 9
  OCaml and OxCaml 9
  Memory 10
Using it 11
  Rust 11
  OCaml and OxCaml 12
  The phidentify command 14
  The C ABI 14
The identities 16
Whole-file SHA-256 17
  What goes in 17
  What equality proves 17
  What it does not prove 17
  When to use it 17
  Reporting 17
  Cost 17
DNG encoded raw blocks 18
  What goes in 18
  What equality proves 18
  What it does not prove 18
  When to use it 19
  Reporting 19
  Cost 19
Image data 20
  What goes in 20
  What equality proves 20
  What it does not prove 21
  When to use it 21
  Why DNGs are refused 21
  Deciding from the first bytes 21
  Reporting 21
  Cost 22
Block marks and skeletons 23
  Blocks 23
  The block-marks identity 23
  Skeletons 24
Lightroom Classic original fingerprint 25
  What it is 25

- What goes in ... 25
- Supported strata ... 25
- Qualification ... 25
- What equality proves ... 26
- What it does not prove ... 26
- When to use it ... 26
- Reporting ... 26
- Cost ... 26

Metadata record ... 27
- What goes in ... 27
- What equality proves ... 27
- What it does not prove ... 27
- When to use it ... 27
- Compact records and diffs ... 27
- Reporting ... 28
- Cost ... 28
- Licence ... 28

Preserved digest ... 29
- What it is ... 29
- Where the region comes from ... 29
- The formula ... 29
- Cross-check with Semantic Unison ... 30
- What equality proves ... 30
- What it does not prove ... 30
- When to use it ... 30
- Reporting ... 30
- Cost ... 31

The identities side by side ... 32

Which identity answers which question ... 33
- Is this file in my backup? ... 33
- Was this only a metadata edit? ... 33
- Where is this file after an editor wrote into it? ... 33
- Can I get the file back as it was before the edit? ... 33
- Is this the same shot after a capture-time shift? ... 33
- Is this still the original my catalog recorded? ... 34
- Did the shuttle copy arrive intact? ... 34
- Can I delete this copy? ... 34

Testing ... 35

Licences ... 36

Reference ... 37

About this manual

This manual is by Alexy Khrabrov, with Claude as co-author. The ideas and decisions are Alexy's; Claude wrote the text and the code with him.

It describes phidentify 0.3.0 as of 2026-11-10. phidentify is licensed under the MIT License; some optional builds link code under other licences, as the chapter on licences explains. Its source is at github.com/firstpair/phidentify.

Every example in this manual uses synthetic files or generic paths. Nothing here depends on a particular photo library.

What phidentify is

"Are these two photos the same?" has no single answer. Two files can hold the same bytes, or the same encoded raw data with different keywords, or different bytes that Lightroom Classic would still accept as the same original. A whole-file checksum answers the first question and is silent on the others. A metadata comparison answers a fourth question and says nothing about pixels.

phidentify computes several **named identities** for a photo file, each answering one question:

Identity	Rust field	OCaml field	Equal values mean
Whole-file SHA-256	`sha256`	`sha256`	every byte is equal
DNG encoded raw blocks	`dng_raw_blocks_sha256`	`dng_raw_blocks`	the same encoded primary raw data, under the same encoding tags
Lightroom Classic original fingerprint	`lightroom_digest`	`lightroom`	Lightroom Classic's original matcher would pair the files
Metadata record	`metadata_digest`	`metadata`	equal exiftool-rs metadata records, file location left out
Preserved (complement) digest	`preserved_sha256`, with `dng_semantic_profile`	`preserved`, with `dng_semantic`	every byte outside the primary XMP packet is equal, at the same size and layout
Image data	`image_data_sha256`	`image_data`	the same stored image bytes, in a camera raw, TIFF or JPEG that is not a DNG
Block marks	`block_marks_sha256`	`block_marks`	the same large blocks, by length and sampled ends, wherever they sit in the file

Beside the identities, phidentify can hand back a file's **skeleton**: every byte outside those large blocks, with a map of where each block goes. A skeleton is not an identity. It is what rebuilds the file as it is now from a later version that a metadata edit has rewritten. The chapter on block marks and skeletons covers both.

Three rules hold for every identity, in every interface:

1. **They are separate.** One identity never stands in for another. Equal raw blocks are not equal bytes; equal image data is not equal metadata; an equal Lightroom fingerprint is not equal pixels; an equal metadata record says nothing about image data.
2. **A missing identity is not a match.** An identity that was not requested, does not apply to the file, or failed, is reported as such. Two files that both failed an identity have not matched on it.
3. **Every value comes from one consistent read.** phidentify checks that the file did not change while it was identified. If it did, the whole result is an error.

Where it fits

This book is one of nine on the FirstPair photo tools. Each owns one part, and The FirstPair Photo Stack shows how they fit together.

- **tiffgraph** walks a TIFF or DNG file's structure under a fixed grammar and finds its editable XMP packet.

- **phidentify** computes a photo's identities: SHA-256, DNG raw blocks, image data, block marks, Lightroom fingerprint, metadata record and preserved digest, and the skeleton that rebuilds a file from an edited copy.
- **phindex** keeps those identities and a skeleton for every file of a volume, aligns two volumes by content, and restores a file as it was before a metadata edit.
- **DNG Compare** compares pairs and preserves one side byte for byte as deltas against the other, restorable even after the other side is edited, with a searchable metadata section, through `dngc`.
- **Semantic Unison** synchronizes photo trees, sending metadata edits without resending image bytes, keeps the versions it replaces as small skeletons, and preserves what a shuttle disk brings into the same archives.
- **packdir** keeps directories of small files as SQLite archives that still work like plain files.
- **Tursolite** is the SQLite and Turso layer under packdir, DNG Compare and Mechanic.
- **diskproxy** serves a folder as a virtual disk, such as previews with camera metadata spliced in, and captures ratings into sidecars.
- **Lightroom Repair** repairs Lightroom Classic catalogs, previews and Sync without forging their state.

How phidentify serves them

phidentify is one of three sister projects that grew out of the same photo-archive work. Each owns a different piece.

- **dng-compare** compares DNG archives: it pairs files across two trees, compares their raw samples, decodes when encodings differ, reports metadata differences, and preserves exact older versions as originals or verified deltas. phidentify's raw-block identity is dng-compare's `dng-subset` block comparison turned into a digest, and its metadata identity uses dng-compare's exiftool-rs engine. Its manual is on the First Pair shelf.
- **tiffgraph** is a pure-OCaml, bounded TIFF/DNG graph walker for stock OCaml and OxCaml. Under a fixed grammar it finds a DNG's primary XMP packet, checks that nothing else the file declares lies inside it, and refuses anything outside the grammar with a reason. phidentify uses it to locate the region the preserved digest leaves out. Its manual is on the First Pair shelf.
- **Semantic Unison** is a fork of the Unison file synchronizer that synchronizes DNG metadata separately from image bytes. Peers negotiate which tiffgraph grammar they trust, as the profiles `org.firstpair.dng-xmp/0`, `/1` and `/2`, and Unison owns the `SDE/0` definitions and proofs that a metadata transfer left every other byte alone. phidentify reproduces Unison's `SDE/0-preserved` hash so that an index built outside Unison speaks the same identity. Its manual is on the First Pair shelf.

phidentify itself computes identities and nothing else. It does not compare trees, write files, touch catalogs, or synchronize anything.

Comparing trees is the job of its main consumer, **phindex**, which keeps every phidentify identity of every file under a directory tree in a resumable SQLite index. Since its version 0.2.0, `phindex align` accounts for the files of one indexed archive in another by these identities rather than by name, so copies that were moved, renamed, re-dated or had other metadata edited are still found. phindex also uses the image-data prefix test described below to add the image-data identity to an existing index without reading most files again. Its manual is on the First Pair shelf.

What it is not

- **Not a rendering identity.** No identity here says that two files look the same after processing. Crop, white balance, profiles and develop settings live in metadata, and the raw-data identities ignore them.

- **Not a decoded-pixel identity**, except inside the Lightroom fingerprint, which decodes for its own purpose and reports only Lightroom's value.
- **Not a validator.** A file that yields every identity is not thereby a correct DNG.
- **Not Lightroom.** The Lightroom fingerprint is an independent reimplementation, qualified against values Lightroom Classic stored. It does not call Adobe code.

Architecture

phidentify is a Rust core with a C ABI, and an OCaml package over that ABI.

```
               Rust core (rust/phidentify)
sha256 · dng_blocks · image_data · lightroom · metadata
      semantic (tiffgraph Rust crate) · preserved
                        |
            C ABI (rust/phidentify-ffi, ocaml/lib/phidentify.h)
                        |
     OCaml package `phidentify` (ocaml/lib)  ── tiffgraph
        stock OCaml 5: Domain.spawn pool
        OxCaml:        Domain.Safe.spawn pool, portable closures
```

The Rust core (`rust/phidentify`, edition 2024, pinned to Rust 1.99.0) holds every identity. `identify(path, Options)` reads the file once and takes every requested identity from the same bytes. With its `semantic` feature it also walks a DNG with tiffgraph's Rust crate, to find the region for the preserved digest itself. The crate forbids `unsafe` code.

The C ABI (`rust/phidentify-ffi`) exposes three functions, declared in `ocaml/lib/phidentify.h`. One call returns every requested identity as a single buffer. It is built as a static library, `libphidentify_ffi.a`, and also as an rlib.

The OCaml package (`ocaml/`, opam name `phidentify`) binds that library and adds two things of its own:

- its own **tiffgraph walk**, with the pure-OCaml tiffgraph, that locates a DNG's primary XMP packet under the strictest grammar that accepts it and supplies the regions for the preserved digest. The C ABI does not run the Rust walk; the OCaml walk and the Rust walk use the same grammars in the same order, with the same packet limit, and give the same profiles, regions and refusal reasons;
- a **domain pool** that identifies many files in parallel and returns results in input order.

The pool is compiled for the switch it is built in. On **stock OCaml 5** it uses `Domain.spawn`. On **OxCaml** it uses `Domain.Safe.spawn`, which accepts only portable closures; the externals it calls carry the `@@ portable` modality, and the paths are shared as an immutable array. dune selects the OxCaml pool when the `stdlib_stable` library exists, which only OxCaml switches provide, and the stock pool otherwise. `Phidentify.backend` reports which one was compiled in: `"domains-stock"` or `"domains-oxcaml"`.

In both pools the tiffgraph walk runs **inside the worker**. Each worker opens the file read-only through a C stub, builds a tiffgraph reader over positional reads on that descriptor, walks, closes, and then calls the Rust identification. Nothing non-portable crosses domains: under OxCaml, tiffgraph's own interface is `@@ portable`, and the reader is built inside the worker rather than captured from outside. The C stubs release the OCaml runtime lock while the Rust core hashes and while the walker's reads run, so the domains really do work in parallel, under stock OCaml and under OxCaml alike.

Building

Rust

The Rust toolchain is pinned in `rust/rust-toolchain.toml` to 1.99.0, with rustfmt and clippy; rustup installs it on first use.

```
git clone https://github.com/firstpair/phidentify.git
cd phidentify/rust
cargo build --release                                  # sha256, raw blocks, image
data, Lightroom, DNG semantic
cargo build --release --features phidentify/metadata   # adds the metadata record
cargo test --release
```

The `lightroom` feature is on by default and links rawler 0.8.0 to decode raw samples. The `semantic` feature is on by default too; it fetches the Rust crate of tiffgraph from GitHub at a pinned commit and lets the core walk DNGs for the preserved digest. The `metadata` feature is off by default; it fetches dng-compare's `dng-exiftool-ffi` crate from GitHub at a pinned commit, and it changes the licence of the resulting binary (see the chapter on licences). Build without default features to get only the whole-file, raw-block and image-data identities, and the preserved digest over regions the caller supplies:

```
cargo build --release -p phidentify --no-default-features
```

`make test` at the repository root runs the Rust tests, clippy with warnings as errors, and the OCaml tests in the current switch.

OCaml and OxCaml

The OCaml package needs OCaml 5.1 or later (stock OCaml or OxCaml), dune 3.9 or later, and tiffgraph. Its opam file pins tiffgraph to a commit of `firstpair/tiffgraph`; outside opam, pin it yourself:

```
opam pin add tiffgraph git+https://github.com/firstpair/tiffgraph.git#main
```

Build the Rust static library first, then the OCaml package, in the switch you want:

```
(cd rust && cargo build --release -p phidentify-ffi)
cd ocaml
opam exec --switch default  -- dune build && opam exec --switch default  -- dune test
--force
opam exec --switch 5.2.0+ox -- dune build && opam exec --switch 5.2.0+ox -- dune test
--force
```

The switch names are examples: use your own stock OCaml switch and your own OxCaml switch. Test in both if you change anything the pools touch. The test prints the version and the backend, for example `ok phidentify 0.3.0 +lightroom domains-stock` under stock OCaml and `ok phidentify 0.3.0 +lightroom domains-oxcaml` under OxCaml.

To include the metadata identity in the OCaml package, build the FFI library with it: `cargo build --release -p phidentify-ffi --features metadata`. `Phidentify.version ()` then ends in `+metadata`.

Two environment variables control how the Rust library is linked:

- `PHIDENTIFY_FFI_LIBDIR` names the directory holding the Rust artifacts. The default is `../rust/target/release`, relative to the `ocaml/` project.
- `PHIDENTIFY_FFI_LINK` is `static` (the default) or `dynamic`. In static mode the archive is copied into the build tree as a tracked file, so a rebuilt Rust library relinks the executables, and it is installed beside the OCaml library with `phidentify.h`. In dynamic mode the shared library is linked with an rpath to its directory.

Memory

The C library, `phidentify-ffi`, uses mimalloc as the global allocator for Rust allocations. Under the macOS system allocator, metadata extraction through exiftool-rs fragmented the heap and kept about 0.5 MB resident per file in long multi-domain runs; with mimalloc it stays flat. The Rust crate itself leaves the allocator to the program that links it. To measure peak memory one identity at a time, run the probe example: `cargo run --release --example memprobe -- MODE FILE...`, where `MODE` is `sha`, `blocks`, `lightroom`, `metadata`, `image` or `all`.

Using it

Rust

```rust
use phidentify::{identify, Options};
use std::path::Path;

let opts = Options { dng_blocks: true, lightroom: true, image_data:
true, ..Options::default() };
let id = identify(Path::new("photo.dng"), opts)?;
println!("{}", id.sha256);
match &id.dng_raw_blocks_sha256 {
    None => println!("raw blocks: not attempted"),
    Some(Ok(digest)) => println!("raw blocks: {digest}"),
    Some(Err(reason)) => println!("raw blocks failed: {reason}"),
}
```

`Options` has one switch per optional identity, and derives `Default` with every switch off:

```rust
pub struct Options {
    pub dng_blocks: bool,
    pub lightroom: bool,
    pub metadata: bool,
    /// Editable regions found by a tiffgraph walk; when given, the
    /// `SDE/0-preserved` complement digest is computed over the same bytes.
    pub preserved_regions: Option<Vec<(u64, u64)>>,
    /// Walk the bytes with tiffgraph (strictest of dng-xmp/0, /1, /2)
    /// and take `preserved_regions` from it; needs the `semantic` feature.
    pub dng_semantic: bool,
    /// Image-data identity of TIFF-family raws, TIFFs and JPEGs (not DNGs).
    pub image_data: bool,
    /// The block-marks identity.
    pub block_marks: bool,
    /// Keep the file's skeleton in the result.
    pub skeleton: bool,
}
```

The whole-file SHA-256 is always computed. `identify` returns `Result<Identity>`:

```rust
pub struct Identity {
    pub stamp: file::Stamp,
    pub sha256: String,
    pub dng_raw_blocks_sha256: Outcome,
    pub lightroom_digest: Outcome,
    pub metadata_digest: Outcome,
    pub preserved_sha256: Outcome,
    pub dng_semantic_profile: Outcome,   // Ok(profile) of the accepting grammar
    pub image_data_sha256: Outcome,
    pub block_marks_sha256: Outcome,
    pub skeleton: Option<Result<skeleton::Skeleton, String>>,   // not serialized
```

```
    pub metadata_json: Option<String>,   // not serialized
}
pub type Outcome = Option<Result<String, String>>;
```

`Outcome` has three states: `None` when the identity was not attempted, `Some(Ok(value))`, and `Some(Err(reason))` when it failed or does not apply. The chapter on reporting lists exactly which happens when.

For the preserved digest a Rust caller has two choices:

- Set `dng_semantic: true`. The core walks the file with tiffgraph's Rust crate, using the same three grammars in the same order as the OCaml package, records the accepting grammar in `dng_semantic_profile`, and computes `preserved_sha256` over the packet it found. `phidentify::semantic::strictest(bytes)` is that walk over bytes in memory. For a file over the in-memory limit, the core walks it from disk instead, with tiffgraph's positional `FileReader` (`semantic::strictest_file(path)`), as the OCaml package's file reader does, so both report the same profile or the same refusal; the preserved digest itself then fails with `file over the in-memory limit`. When the walk refuses the file, `dng_semantic_profile` holds the reason and `preserved_sha256` is `None`.
- Leave `dng_semantic` off and pass the regions in `Options::preserved_regions`, as sorted, disjoint `(offset, length)` pairs inside the file, obtained from a tiffgraph walk or from Semantic Unison. `phidentify::file::preserved_sha256(bytes, regions)` computes the same digest over bytes already in memory.

The `identify` example requests every identity the build supports, including the DNG semantic walk and image data, and prints one JSON line per argument:

```
cargo run --release --example identify -- photo.dng notes.txt
cargo run --release --features phidentify/metadata --example identify -- photo.dng
```

On a synthetic text file, a 14-byte TIFF with one empty directory and a missing path it prints (without the stamp, the digests shortened with …):

```
{"identity":{"dng_raw_blocks_sha256":null,"dng_semantic_profile":{"Err":"not TIFF"},
  "image_data_sha256":{"Err":"not a TIFF-family file or
JPEG"},"lightroom_digest":null,
"metadata_digest":null,"preserved_sha256":null,"sha256":"c5e496e7…"},"path":"notes.txt"}
{"identity":{"dng_raw_blocks_sha256":{"Err":"TIFF offset outside file"},
  "dng_semantic_profile":{"Err":"missing DNGVersion"},"image_data_sha256":{"Err":"no
image data in TIFF"},
"lightroom_digest":null,"metadata_digest":null,"preserved_sha256":null,"sha256":"5b8de6f0…"},"path":"tiny.t
{"error":"stat missing: No such file or directory (os error 2)","path":"missing"}
```

In JSON, `null` is not attempted, `{"Ok": …}` is a value and `{"Err": …}` is a failure. `preserved_sha256` stays `null` when the walk refuses the file; for a DNG a grammar accepts, it holds the digest.

OCaml and OxCaml

The OCaml interface is the same under stock OCaml and OxCaml.

```
type outcome = Not_attempted | Ok of string | Failed of string

type identity = {
  sha256 : string;
  dng_raw_blocks : outcome;
  lightroom : outcome;
  metadata : outcome;
  dng_semantic : outcome;  (* Ok profile *)
  preserved : outcome;
  metadata_record : string option;
  image_data : outcome;
  block_marks : outcome;
  skeleton : (stored, string) result option;
}
and stored = { record : string; packed : string }

type request = { dng_raw_blocks : bool; lightroom : bool; metadata : bool;
                 dng_semantic : bool; compact_record : bool; image_data : bool;
                 block_marks : bool; skeleton : bool }

val all : request
val skeleton : sha256:string -> string -> outcome * (stored, string) result option
val no_skeleton_walker : string
val image_data_candidate : string -> bool
val image_data_prefix_len : int
val image_data_not_applicable : string
val image_data_dng : string
val version : unit -> string
val backend : string
val recommended_workers : unit -> int
val identify : ?request:request -> string -> (identity, string) result
val identify_all : ?request:request -> workers:int -> string list -> (identity,
string) result list
```

`all` requests every identity, `image_data` and `block_marks` included, and leaves `compact_record` and `skeleton` off. A request field that the linked library does not support, such as `metadata` without the `metadata` feature, comes back as `Failed "built without the metadata feature"`, not as `Not_attempted`.

`dng_semantic = true` runs the tiffgraph walk and, when it succeeds, the preserved digest over its region. `dng_semantic` then holds the name of the grammar that accepted the file, and `preserved` the digest.

```
match Phidentify.identify "photo.dng" with
| Error reason -> prerr_endline reason
| Ok id ->
  print_endline id.sha256;
  (match id.dng_semantic, id.preserved with
   | Ok profile, Ok digest -> Printf.printf "%s %s\n" profile digest
   | Failed reason, _ -> Printf.printf "no preserved digest: %s\n" reason
   | _ -> ())
```

`image_data` is the image-data identity, reported like the others. `image_data_candidate`, `image_data_prefix_len` and the two reason strings are the OCaml counterparts of the Rust

`image_data::candidate`, `PREFIX_LEN`, `NOT_APPLICABLE` and `DNG`, and give the same answers; the chapter on image data explains how an indexer uses them.

`identify_all ~workers paths` identifies a list in parallel and returns the results in input order. It raises `Invalid_argument` when `workers < 1`, and it uses at most `min workers (List.length paths)` domains, capped at `recommended_workers ()`.

The phidentify command

The OCaml package installs `phidentify [-j N] FILE...`. It prints one tab-separated line per file, in argument order:

```
path  sha256  dng_raw_blocks  lightroom  metadata  dng_semantic  preserved  image_data
block_marks
```

Each identity column is the value, `-` when not attempted, or `!reason` when it failed. The command never requests the metadata identity, so that column is always `-`. A file that cannot be identified at all gets a two-column line, `path`, then `!reason`. `-j N` sets the number of workers; the default is `recommended_workers ()`.

On the same synthetic text file and TIFF:

```
notes.txt  c5e496e7…  -                          -  -  !not TIFF           -  !not a
TIFF-family file or JPEG  !no skeleton walker for this format
tiny.tif   5b8de6f0…  !TIFF offset outside file  -  -  !missing DNGVersion  -  !no
image data in TIFF           !no blocks
```

The C ABI

```c
#define PHIDENTIFY_DNG_BLOCKS 1u
#define PHIDENTIFY_LIGHTROOM 2u
#define PHIDENTIFY_METADATA 4u
#define PHIDENTIFY_COMPACT_RECORD 8u
#define PHIDENTIFY_IMAGE_DATA 16u
#define PHIDENTIFY_BLOCK_MARKS 32u
#define PHIDENTIFY_SKELETON 64u
int32_t phidentify_identify(const uint8_t *path, size_t path_len, uint32_t flags,
                            const uint64_t *regions, size_t n_regions,
                            uint8_t **out, size_t *out_len);
int32_t phidentify_skeleton(const uint8_t *path, size_t path_len,
                            const uint8_t *sha256, size_t sha256_len,
                            uint8_t **out, size_t *out_len);
int32_t phidentify_version(uint8_t **out, size_t *out_len);
void phidentify_free(uint8_t *ptr, size_t len);
```

`path` is a byte string of `path_len` bytes, not NUL-terminated. `regions` holds `n_regions` pairs of `(offset, length)`, or is `NULL` when no preserved digest is wanted.

`phidentify_identify` returns a status:

Status	out holds
0	the identities

Status	out holds
1	the reason the file could not be identified
2	nothing; an argument was invalid
3	a message; the Rust core panicked, and the panic was caught

With status 0, `out` holds ten fields separated by NUL bytes, in this order: sha256, DNG raw blocks, Lightroom digest, metadata digest, preserved digest, metadata record JSON, image data, block marks, skeleton record JSON, packed skeleton in hex. The image-data field is filled only when `PHIDENTIFY_IMAGE_DATA` (16) is set in `flags`, block marks only with `PHIDENTIFY_BLOCK_MARKS` (32), and the two skeleton fields only with `PHIDENTIFY_SKELETON` (64). New fields are added at the end, so that earlier ones keep their places; a caller built against seven fields must be rebuilt. Each field is `ok:<value>`, `err:<reason>`, or empty when not attempted. Free every `out` buffer exactly once with `phidentify_free(out, out_len)`.

`phidentify_skeleton` returns the last three of those fields (block marks, skeleton record, packed skeleton) for a file whose content has the given SHA-256. It reads the file's structures only, never the whole file, and is for adding block marks and a skeleton to a file that was identified earlier.

`phidentify_version` returns a string such as `phidentify 0.3.0 +lightroom +metadata`, naming the features the library was built with.

The identities

The six chapters that follow describe each identity in the same order: what goes in, what equality proves and what it does not, when to use it, how it reports failure and not-applicable, and what it costs.

Whole-file SHA-256

What goes in

Every byte of the file, in order, through SHA-256. The value is 64 lower-case hex digits. It is always computed.

What equality proves

Equal values mean the two files are byte-for-byte identical, up to the collision resistance of SHA-256. Nothing else in phidentify is this strong.

What it does not prove

It does not say anything useful about files that differ. A changed rating, a rewritten preview or one flipped byte of padding all change it, and it cannot tell those apart from a different photograph. File names, dates, permissions and extended attributes are not part of it.

When to use it

Use it when the question is about bytes: is this backup copy exact, did the copy across a network or onto a portable drive arrive intact, is this the file I recorded last year.

Reporting

There is no not-applicable case. If the SHA-256 cannot be computed, the file cannot be identified at all, and the whole call fails: the Rust `identify` returns `Err`, the OCaml functions return `Error reason`, the C ABI returns status 1, and the command prints `path`, then `!reason`. The call also fails if the path is not a regular file (symbolic links are refused) or if the file changes while it is read.

The change check uses a **stamp** taken before and after identification: size, modification time, change time and inode. The device number is recorded but left out of the comparison, because it changes when a volume is remounted. The stamp is returned in the Rust `Identity`.

Cost

One sequential read of the file and one SHA-256 pass. Files up to 1 GiB are read once into memory, and every other identity is computed from those same bytes. Larger files are hashed in 4 MiB chunks. The identities that need the whole file in memory are then not available: raw blocks and the Lightroom fingerprint are not attempted, and the preserved digest and image data fail with `file over the in-memory limit`. The DNG semantic walk still runs, from disk, and the metadata record, which its engine reads from the path, is still computed.

DNG encoded raw blocks

What goes in

The encoded primary raw images of a DNG, exactly as stored, without decoding. phidentify walks the TIFF or BigTIFF graph: the chain of directories from the header and every SubIFD. A directory is a **primary raw image** when its NewSubfileType is 0 (or absent) and its photometric interpretation is WhiteIsZero, BlackIsZero, RGB, CFA or LinearRaw. The file must also carry a DNGVersion tag; a TIFF without one is not a DNG.

For each primary raw image, phidentify takes:

- its **encoding-tag signature**: the byte order, then, for each tag present from a fixed list of 30 encoding tags, its number, type, count, length and stored value bytes. The list covers dimensions, bits and sample format, compression, photometric interpretation, strip and tile geometry, the predictor, and the JPEG tables and parameters, plus a few DNG encoding tags; it is the `ENCODING` list in `dng_blocks.rs`;
- its **block counts**: how many strips or tiles, and the byte count of each;
- its **block bytes**: the content of every strip or tile, in order.

The digest is SHA-256 over:

```
"PHIDENTIFY-DNG-ENCODED-RAW-BLOCKS-1\0"
‖ byte-order byte (1 for big-endian)
‖ u64le number of primary raw images
‖ for each image, in walk order:
     u64le signature length ‖ signature
   ‖ u64le block count
   ‖ for each block: u64le byte count ‖ block bytes
```

The domain string names the definition. If what it covers ever changes, the domain string changes with it, because every stored digest would change.

What equality proves

Two DNGs have equal raw-block digests exactly when dng-compare's `dng-subset` block comparison finds them equal: the same byte order, the same number of primary raw images, and for each image the same encoding-tag signature, block counts and block bytes. The stored, encoded raw data is the same.

What it does not prove

- **Not metadata equality.** XMP, keywords, ratings, develop settings, Exif fields, maker notes and previews do not enter it. Neither do interpretation tags outside the encoding list, such as black and white levels, the CFA pattern, linearization tables, color matrices and the default crop.
- **Not placement.** Block offsets are left out, so a file whose blocks moved when its metadata grew keeps the same digest. That is the point.
- **Not decoded-pixel equality.** The same samples re-encoded with different compression, tiling or byte order give a different digest. To compare decoded samples across encodings, use dng-compare, which decodes when it has to.
- **Not rendering equality**, and not Lightroom's fingerprint.

When to use it

Use it to recognize the same raw data under changed metadata: a DNG whose keywords were written into the file, a copy whose preview was refreshed, a backup taken before a rating changed. A raw-block match tells you the photograph's encoded data is present in both files. It does not tell you either file is a complete backup of the other.

Reporting

Situation	Rust	OCaml	Command
not requested	`None`	`Not_attempted`	`-`
file does not start with a TIFF header	`None`	`Not_attempted`	`-`
file over 1 GiB	`None`	`Not_attempted`	`-`
TIFF without DNGVersion or without a primary raw image	`Some(Err("No DNG raw image found"))`	`Failed`	`!No DNG raw image found`
malformed graph, offsets or blocks outside the file	`Some(Err(reason))`	`Failed reason`	`!reason`
success	`Some(Ok(hex))`	`Ok hex`	the digest

The walk is bounded: at most 4,096 directories, 8,192 entries per directory, 16 MiB per selected tag value and a million integers per array. A file beyond those bounds fails; it is never truncated into a match.

Cost

Cheap. The walk reads only the directories and the selected tags, and the hash reads the raw blocks once, from the bytes already in memory. There is no decoding. For a typical DNG the cost is close to one SHA-256 pass over the raw data.

Image data

What goes in

The stored image bytes of a camera raw, TIFF or JPEG that is not a DNG, and nothing else. Which bytes those are depends on the format, which phidentify recognizes from the first bytes of the file:

- **TIFF family.** Classic TIFF and BigTIFF, and the camera raws built on TIFF: NEF, CR2, ARW, 3FR, PEF, ORF, the RW2 header and others. phidentify walks the chain of directories from the header and every SubIFD, in walk order, and takes from **every** directory that has them the strip or tile payloads named by its offsets and byte counts (TileOffsets and TileByteCounts, or StripOffsets and StripByteCounts). Every directory counts, not just the primary raw image: thumbnails and previews stored as strips or tiles are part of it.
- **Panasonic RW2.** Its raw data has no byte counts. When a directory has neither strip nor tile offsets but has RawDataOffset (tag 280), the span from that offset to the end of the file is taken.
- **Fujifilm RAF.** The header, which starts with `FUJIFILMCCD-RAW`, names the CFA section by its offset and length, big-endian at bytes 100 and 104. That section is taken.
- **JPEG.** The primary image's frame, tables and entropy-coded scans: every segment after SOI up to EOI except the APPn segments (EXIF, XMP, ICC profiles, thumbnails, MPF) and comments. Trailing data after EOI, where some cameras put further images, is left out.

The digest is SHA-256 over a domain string, a format letter and the bytes in a fixed framing. All integers are unsigned 64-bit little-endian:

```
"PHIDENTIFY-IMAGE-DATA-1\0"
‖ "T" ‖ byte-order byte (1 for big-endian)                    -- TIFF family, RW2
     ‖ for each directory with image data, in walk order:
         u64le directory number (1, 2, …) ‖ u64le block count
       ‖ for each strip or tile: u64le byte count ‖ bytes
‖ "F" ‖ u64le CFA length ‖ CFA bytes                           -- RAF
‖ "J" ‖ for each segment after SOI, up to EOI, except APPn and COM:
         the segment as stored (marker, length, body)
       ‖ after each SOS segment: u64le scan length ‖ scan bytes
     ‖ FF D9                                                   -- JPEG
```

An RW2's raw span is framed as one directory with one block. A JPEG's scan bytes are hashed as stored, with their stuffed `FF 00` bytes and restart markers; standalone restart markers and fill bytes between segments are skipped. Offsets are never hashed, so image data that moved inside the file keeps the digest.

The domain string names the definition. If what it covers ever changes, the domain string changes with it.

What equality proves

Equal digests mean the two files store the same image bytes, in the same order and the same framing: the same strips and tiles, the same raw span, the same CFA section, or the same JPEG frame, tables and scans. The photograph as the camera or converter encoded it is the same.

On real files, an ExifTool capture-time shift changed the whole-file SHA-256 of TIFF, ARW, RW2 and JPEG files and left their image data equal. (ExifTool cannot write 3FR, so that format was not part of the check.)

What it does not prove

- **Not metadata equality.** EXIF, maker notes, XMP, IPTC, ICC profiles and comments are left out, and so are directory entries that do not name image data. Two files with equal image data can have different capture times, keywords and ratings.
- **Not decoded-pixel equality.** The same picture re-encoded, re-compressed, re-tiled or saved with another byte order gets a different digest. So does a file whose preview strips were rewritten, because those strips are image data.
- **Not a raw-block identity.** For DNGs, use the raw-block identity. The two are separate identities with separate domain strings, and a non-DNG file never has a raw-block digest.

When to use it

Use it to match copies of non-DNG camera files and JPEGs whose metadata was edited. The commonest such edit is a capture-time shift, made because the camera clock was wrong or set to another time zone: it rewrites a few bytes of EXIF in every file, every whole-file SHA-256 changes, and a byte comparison says nothing is backed up. With image data, the shifted copy and the original still agree. The same holds for keywords, ratings or copyright written into a JPEG's APP segments.

Why DNGs are refused

A DNG is a TIFF, and its strips include its previews. Lightroom Classic regenerates a DNG's previews when it saves metadata into the file, so an image-data digest of a DNG would change on a metadata edit, which is the change it is meant to survive. The raw-block identity already answers the question for DNGs, over the primary raw images only. So a TIFF-family file with a DNGVersion tag in any directory is refused with the reason `DNG: the raw-block identity applies instead`.

CR3, HEIC and other ISO-BMFF or unknown containers are refused as not applicable.

Deciding from the first bytes

Whether a file can have image data is decided by its first 16 bytes. `image_data::candidate(prefix)` in Rust and `Phidentify.image_data_candidate` in OCaml take the first `image_data::PREFIX_LEN` (`image_data_prefix_len`, 16) bytes of a file, or all of a shorter file, and return true when they start like a JPEG (`FF D8 FF`), a RAF (`FUJIFILMCCD-RAW`) or a TIFF-family file (`II` or `MM`, with at least four bytes).

When `candidate` is false, the full identity fails with exactly `image_data::NOT_APPLICABLE` (`image_data_not_applicable`), `not a TIFF-family file or JPEG`, so a caller can record that outcome without reading the rest of the file. When it is true, only a full read can tell: a DNG fails with `image_data::DNG` (`image_data_dng`), and a malformed file with its own reason.

This exists for indexers. When phindex 0.2.0 added the image-data column to existing indexes, it settled rows that already had a raw-block digest with the DNG reason without opening the file, read only the first 16 bytes of every other unchanged file, recorded the not-applicable reason for videos, sidecars and other non-candidates, and read in full only the files that can have image data.

Reporting

Situation	Rust	OCaml	C field
not requested	`None`	`Not_attempted`	empty

Situation	Rust	OCaml	C field
first bytes are not a candidate	`Some(Err("not a TIFF-family file or JPEG"))`	`Failed` with the same reason	`err:…`
DNG	`Some(Err("DNG: the raw-block identity applies instead"))`	`Failed` with the same reason	`err:…`
file over 1 GiB	`Some(Err("file over the in-memory limit"))`	`Failed` with the same reason	`err:…`
malformed file	`Some(Err(reason))`	`Failed reason`	`err:reason`
success	`Some(Ok(hex))`	`Ok hex`	`ok:hex`

Typical malformed-file reasons are `unsupported TIFF header`, `image data without byte counts`, `image data outside file`, `no image data in TIFF`, `RAF CFA data outside file`, `JPEG ends before EOI` and `JPEG without a scan`. The TIFF walk has the same kind of bounds as the raw-block walk: at most 4,096 directories, 8,192 entries per directory and a million values per array. A file beyond them fails; it is never truncated into a match.

Cost

Cheap. The walk reads only the directories and the offset and count tags, and the hash reads the image bytes once, from the bytes already in memory. There is no decoding. For a raw file the cost is close to one SHA-256 pass over the file.

Block marks and skeletons

Blocks

A photo file is a small amount of structure wrapped around a few large blocks. In a DNG the structure is the TIFF header, the image file directories with their tags, and the XMP packet; the blocks are the raw image's strips or tiles, the embedded previews, the maker notes and the camera profiles. When an editor writes a rating, a caption or develop settings into the file, it rewrites the structure and carries the blocks over unchanged, often to other offsets because the XMP packet grew.

phidentify cuts a file into **blocks** and the **skeleton**, which is every byte outside the blocks, in file order. The cutting rules are those of dng-compare's archive format (its specification, section 13.1), so a skeleton made here is the one `dngc` makes. In short:

- **Classic TIFF** (DNG and the TIFF-based raws). The blocks are the strips or tiles of each image, merged when they lie within 3 bytes of each other; each embedded JPEG stream; any other out-of-line tag value of 512 bytes or more, except XMP (700), IPTC (33723) and Photoshop resources (34377), which editors rewrite; and any run of 64 KiB or more that no tag points at.
- **JPEG.** The blocks are each segment of 512 bytes or more other than APP1 (Exif, XMP), APP13 (IPTC) and COM, and everything from the start of scan to the end of the file.
- **Anything else** up to 1 MiB, such as an XMP sidecar, is its own skeleton, with no blocks. A larger file of another format has no skeleton.

Each block has a **mark**: the first 32 hex digits of the SHA-256 of its first and last 4,096 bytes, or of the whole block when it is no longer than 8,192 bytes.

The block-marks identity

```
SHA-256( "PHIDENTIFY-BLOCK-MARKS-1\0"
       ‖ format ("tiff" or "jpeg") ‖ 0x00
       ‖ u64le(number of blocks)
       ‖ for each block, sorted by length and then mark:
           u64le(length) ‖ mark (32 ASCII hex digits) )
```

Offsets and roles are left out, so blocks that an edit moved keep the digest.

What equality proves. The two files hold blocks of the same lengths whose first and last 4,096 bytes are equal. In practice that means the same photograph with the same previews, maker notes and profiles, whatever was done to its metadata.

What it does not prove.

- **Not byte equality inside a block.** A mark samples a block's two ends. A change in the middle of a long block, such as an edited pixel in an uncompressed strip, keeps the digest. This identity is a strong hint; proof is a rebuild checked by SHA-256, below.
- **Not metadata equality.** Everything an editor rewrites is outside the blocks.
- **It is stricter than raw blocks or image data.** A regenerated embedded preview changes it, where the raw-block identity of a DNG does not.

When to use it. To find the edited version of a file: the same picture after Lightroom or Photo Mechanic wrote into it, at any path. It applies to DNGs and to other TIFF-based raws and JPEGs alike, and it covers the previews and maker notes that the raw-block and image-data identities leave out.

Reporting. A file stored whole or of a format without a walker fails with `no skeleton walker for this format`. A TIFF or JPEG with no block of 512 bytes or more fails with `no blocks`.

Measured. On 249 real files, each compared before and after years of edits in Lightroom Classic and Photo Mechanic (120 DNGs, 78 JPEGs, and 51 DNGs chosen because their byte differences were large), the block marks were equal in all 249.

Skeletons

A skeleton is the file's format, its size, its blocks (`at`, `len`, `mark`, `role` each) and the bytes outside them. Building one reads the file's structures, those bytes, and each block's two ends; it never reads a whole block, so it costs a handful of small reads however large the file is.

```
use phidentify::skeleton;
let old = skeleton::build(path)?;                // or build_bytes(&bytes)
// ... the file is edited: a rating, an exposure adjustment ...
let mut out = std::fs::File::create("as-it-was.dng")?;
let sha = skeleton::rebuild(&old, path, &mut out)?;
assert_eq!(sha, sha256_before_the_edit);
```

`rebuild` walks the edited file, finds each of the old file's blocks in it by length and mark, and writes the old file in order: skeleton bytes, a block, more skeleton bytes. It returns the SHA-256 of what it wrote. **The caller must compare that with the old content's SHA-256**, because a mark covers only a block's ends. When a block is missing from the edited file, `rebuild` fails and says which.

A skeleton is stored as two values:

- `Skeleton::pack()` gives its bytes as one zstd frame (level 19).
- `Skeleton::record(sha256)` gives a JSON record for the content with that SHA-256, written as Python's `json.dumps` writes it, with the members of dng-compare's skeleton record that do not depend on where the file is: `sha256`, `size`, `format`, `blocks`, `skeleton_bytes`, `skeleton_sha256`, `skeleton_format`.
- `Skeleton::from_record(record, packed)` gives the skeleton back, after checking the bytes against the record.

`identify` returns the skeleton when `Options::skeleton` is set, built from the same read as the identities. From OCaml, `request.skeleton` does the same and `identity.skeleton` holds the record and the packed bytes; `Phidentify.skeleton ~sha256 path` builds them from the file's structures alone.

On real files a packed skeleton is about 5 KB for a DNG and about 10 KB for a JPEG, whose Exif segment with its thumbnail is kept whole.

What a skeleton cannot do. It cannot bring back a block the edit destroyed: a regenerated preview, re-encoded image data, a truncated file. For that the old bytes must be kept from a copy that still has them.

Lightroom Classic original fingerprint

What it is

Lightroom Classic stores a fingerprint of each original file in its catalog, in the `digest` column of the `AgLibraryFileDigest` table. phidentify computes that value for the DNGs it supports, so that a file can be checked against the catalog without Lightroom.

It is an **independent reimplementation**. It contains no Adobe code, and it was qualified only by comparing its output with values Lightroom Classic itself had stored.

What goes in

At the level the source shows, the fingerprint has two layers:

1. **A raw-image digest.** The raw samples are decoded with rawler, before linearization, without camera crop or any rendering. The digest is Adobe's NewRawImageDigest, the raw-image digest the DNG specification defines, recomputed here from the decoded samples: the samples are cut into tiles, each tile is hashed with MD5, and the tile digests are hashed again with MD5.
2. **A wrapper.** That raw digest is wrapped, again with MD5, together with streams that describe how the image is framed: the camera model, the default crop size and origin, the user crop when there is one, and a fingerprint stream of the raw image's opcode lists.

The result is 32 upper-case hex digits, as Classic stores it.

The Rust module also exposes `lightroom::compute(path)`, which returns the intermediate values: the raw-image digest, a SHA-256 of the wrapper inputs, and the engine revision name, `native-original-digest-v5`. Those are for diagnosis; the identity is the final digest.

Supported strata

The fingerprint is computed only for DNGs that are:

- integer-sampled,
- single-plane CFA or LinearRaw,
- stored uncompressed or with lossless JPEG compression, in classic TIFF rather than BigTIFF,
- with raw-image opcodes limited to the two implemented, FixVignetteRadial and FixBadPixelsConstant.

Every other file is an error with a reason, never a guessed value. JPEG files, camera-native raw files and DNG strata outside that list are not implemented.

Qualification

On 2026-11-01, 60 randomly chosen DNGs from a Lightroom Classic 15 catalog were fingerprinted. All 60 matched the fingerprints Lightroom Classic had stored for them.

That is the whole evidence for this identity. It covers the strata above, as Lightroom Classic 15 computed them. Extending the fingerprint to another format needs native values from Lightroom for that format first.

What equality proves

An equal digest means Lightroom Classic's original matcher would pair the two files, or would accept the file as the original a catalog entry recorded. Compare it together with the file size, which Classic stores beside it.

What it does not prove

It is native-matcher correspondence, not byte equality and not pixel equality. Two files with equal fingerprints can differ in metadata, previews and layout. MD5 is used because that is what Lightroom uses; the value is a compatibility identity, not a cryptographic one, and should not be relied on against deliberately crafted files.

When to use it

Use it to check a file against a Lightroom Classic catalog: is this the original the catalog thinks it is, which file on disk does this catalog entry correspond to, did a restore bring back the file Classic expects. Read the catalog's stored values from a copy, never from a catalog Classic has open, and never write them.

Reporting

The fingerprint is attempted only when it is requested and the file parsed as a DNG with a primary raw image, the same gate as the raw-block identity. Otherwise it is not attempted (`None`, `Not_attempted`, `-`).

When attempted, it fails with a reason for any unsupported stratum, malformed metadata, a decoder error, decoded dimensions that disagree with the raw directory, or a change to the file while it is read. A library built without the `lightroom` feature reports `built without the lightroom feature` as a failure for every DNG.

Cost

The most expensive identity by far. It reads the file again from its path and fully decodes the raw image, so it costs CPU in proportion to the sensor size and holds the decoded samples in memory, two bytes per sample. Request it only when the question is about Lightroom.

Metadata record

What goes in

The file's metadata as read by dng-compare's **exiftool-rs engine** (`dng-exiftool-ffi`), an in-process Rust metadata reader. The record is one ExifTool-shaped JSON object, following the flags dng-compare's Perl workers use: group-qualified keys (`-G0:1:4`), numeric values (`-n`), nested XMP structures (`-struct`), duplicates kept as `Copy1`, `Copy2` and so on, and unknown tags included. Embedded image payloads such as previews and thumbnails stored as image tags are left out of the record.

Every record carries an engine stamp member, `ExifTool:DngCompareEngine`, naming the engine and its version, for example `exiftool-rs 0.12.0 (ExifTool … tables)`.

The **metadata digest** is SHA-256 over the record serialized with sorted keys, after removing the members that describe where the file sits rather than what it contains: `SourceFile` and every `File:System:*` member (file name, directory, size, dates, permissions).

The engine stamp is **not** removed. It is part of the digest, so records from different engines, or from different versions of the engine, never compare equal. They are separate identities, and records from Perl ExifTool are another identity again.

What equality proves

Equal digests mean the engine produced the same metadata record for both files, apart from location members.

What it does not prove

Nothing about image data: two different photographs can share a record if the engine sees the same fields. Nor does it cover every byte of metadata: fields the engine does not decode, padding, and the bytes of embedded previews are outside it. It is a description of the metadata the engine exposes, not a byte identity of the metadata. To prove that only the metadata packet changed, use the preserved digest.

When to use it

Use it to group files by metadata, to detect metadata-only edits, and, with `diff`, to say exactly which fields changed.

Compact records and diffs

The full record is available too: `metadata_json` in Rust, `metadata_record` in OCaml.

- `metadata::compact(json)` replaces every string member longer than 1 KiB, such as base64 thumbnails and camera-profile tables, by `long-sha256:<hex>:<length>`, the SHA-256 and byte length of the original string. Compact records are for storage. The digest is always taken over the full record, never the compact one. In OCaml, set `compact_record = true` in the request; over the C ABI, pass `PHIDENTIFY_COMPACT_RECORD`. Why 1 KiB: the strings above that size are almost never the fields people search or compare, such as camera, lens, exposure, dates, ratings and keywords. They are payloads, such as embedded thumbnails, profile tables and maker-note blobs. They make up most of a record's bytes.

Kept as a hash and a length, they still show whether two files differ there; DNG Compare's archives keep the bytes themselves.

- `metadata::diff(left, right)` lists the top-level members whose values differ between two records, ignoring location members. Each difference is `(key, left value, right value)`, with `None` on a side where the member is absent. Nested structures are compared as whole values.

Reporting

Situation	Rust	OCaml	Command
not requested	`None`	`Not_attempted`	always -
a file type the engine does not read	`None`	`Not_attempted`	-
built without the `metadata` feature	`Some(Err("built without the metadata feature"))`	`Failed`	—
the engine reports an error	`Some(Err("metadata extraction failed: …"))`	`Failed`	—
success	`Some(Ok(hex))`	`Ok hex`	—

The engine reports a failed file as an `ExifTool:Error` member in its record. phidentify turns that member into a failure. It is never treated as an empty record, and two failed files never share a digest.

The metadata identity is attempted only for a file whose extension the engine reads, by exiftool-rs's own extension table: DNGs and other raws, JPEGs, TIFFs, HEIC, JPEG XL, XMP sidecars, videos and the rest of that table. Any other file is not attempted, so its outcome is `None`, not a failure. A Capture One `.cop` cache proxy, an `.ini` or a `.dat` has no metadata record, and is not reported as a file whose metadata could not be read. The test looks at the name only, so a `.cop` whose first bytes happen to be a JPEG XL stream is still not read: what the file is for, not what its bytes resemble, decides. `metadata::reads(path)` (OCaml `Phidentify.reads_metadata`, C `phidentify_reads_metadata`) makes the test without opening the file.

Cost

Moderate. The engine reads the file again from its path and decodes its metadata structures; the record can be large when it holds long strings, which is why `compact` exists. It does not decode image data.

Licence

The `metadata` feature links exiftool-rs, which is GPL-3.0-or-later. A binary built with it is covered by the GPL. See the chapter on licences.

Preserved digest

What it is

The preserved digest, also called the complement digest, is a hash of every byte of a DNG **outside** its primary XMP packet, with the file's size and the packet's position bound in. Two files with equal preserved digests can differ only inside that packet. It is Semantic Unison's `SDE/0-preserved` hash, which Unison uses to prove that a metadata transfer changed nothing else.

Where the region comes from

The region left out is found by tiffgraph. phidentify's OCaml package, with the pure-OCaml tiffgraph, and its Rust core, with tiffgraph's Rust crate when `Options::dng_semantic` is set, both walk each file with three tiffgraph grammars in order, from strictest to most permissive:

Profile reported in `dng_semantic`	tiffgraph grammar	Semantic Unison profile
`dng-xmp/0`	`v0`	`org.firstpair.dng-xmp/0`
`dng-xmp/1`	`v1`	`org.firstpair.dng-xmp/1`
`dng-xmp/2`	`merge_only`	`org.firstpair.dng-xmp/2`

The first grammar that accepts the file wins, and its name is recorded in `dng_semantic` (OCaml) or `dng_semantic_profile` (Rust). Accepting the file means tiffgraph found the primary XMP packet and checked that no other extent the file declares lies inside it. The maximum packet size accepted is 1 MiB.

The tiffgraph manual describes what each grammar admits. In short, `v0` admits classic TIFF with standard fields; `v1` adds BigTIFF, embedded camera profiles and the DNG 1.7 JPEG XL fields; `merge_only` adds the private fields camera converters write, kept byte-exact and never interpreted.

The formula

With every integer an unsigned 64-bit big-endian number:

```
SHA-256( "SDE/0-preserved\0"
       ‖ u64be size
       ‖ u64be n
       ‖ (u64be off ‖ u64be len)*          -- each of the n regions
       ‖ for each gap: u64be start ‖ u64be len ‖ bytes )
```

`size` is the file size and `n` the number of regions; phidentify passes one region, the packet. The gaps are the stretches before, between and after the regions, so there are always `n + 1` of them, and each is hashed even when it is empty. That binds the packet's position as well as everything around it.

A small worked example, from the test suite: the 22-byte string `HEADER<xmp one>TRAILER` with one region at offset 6, length 9 hashes

```
"SDE/0-preserved\0" ‖ 22 ‖ 1 ‖ 6 ‖ 9 ‖ 0 ‖ 6 ‖ "HEADER" ‖ 15 ‖ 7 ‖ "TRAILER"
```

Replacing bytes inside the region leaves the digest unchanged; changing any byte outside it changes the digest. Regions must be sorted, disjoint and inside the file, or the digest fails.

Cross-check with Semantic Unison

phidentify's implementation was cross-checked byte for byte against Semantic Unison's own `SemanticDelta.inspect` on 60 DNGs: for every file both accepted, both chose the same strictest grammar and produced the same digest, and files without a packet were refused by both. The Rust walk, added later, gave the same profiles, digests and refusals on the same 60 files.

What equality proves

Equal preserved digests, **under the same profile**, mean that every byte outside the primary XMP packet is equal, and that the packet sits at the same offset with the same length. The files can differ only inside the packet.

Digests are comparable only under the same profile. The digest depends on the grammar through the region it returns, and a file accepted by `dng-xmp/2` that a stricter peer refuses has no `dng-xmp/0` digest at all. Compare `dng_semantic` before comparing `preserved`.

What it does not prove

Anything about the XMP packet. Two files with equal preserved digests can have completely different keywords, ratings and develop settings. It also does not cover a file whose packet changed size: a longer packet moves the gaps and changes the digest, even if nothing else changed. That is deliberate. Semantic Unison treats such a rewrite as a different operation.

When to use it

Use it to prove a metadata-only edit: the working copy and the backup differ, and you want to know whether only the XMP packet changed. With equal preserved digests, the backup plus the working copy's packet is the working copy. This is the identity a synchronizer needs before it sends metadata without image bytes.

Reporting

Situation	`dng_semantic`	`preserved`	Command
not requested	`Not_attempted`	`Not_attempted`	`-`, `-`
no grammar accepts the file	`Failed reason`	`Not_attempted`	`!reason`, `-`
accepted	`Ok "dng-xmp/N"`	`Ok hex`	`dng-xmp/N`, digest
accepted, file over 1 GiB	`Ok "dng-xmp/N"`	`Failed "file over the in-memory limit"`	profile, `!…`

A file over 1 GiB is walked from disk, not from memory: the OCaml package through its positional file reader, the Rust core through tiffgraph's `FileReader`. Both give the same profile, or the same refusal, as an in-memory walk of the same bytes; a read past the end of the file is reported as `read outside the bytes` by both. Only the preserved digest itself needs the bytes in memory.

When every grammar refuses, the reason reported is the last one, from `merge_only`. Typical reasons are `not TIFF`, `missing DNGVersion` and `no XMP packet`. A refused file has **no** preserved digest; it is not a match with anything.

From Rust, with `dng_semantic` set, `dng_semantic_profile` and `preserved_sha256` follow the same table: `Some(Err(reason))` and `None` for a refused file, `Some(Ok(profile))` and `Some(Ok(hex))` for an accepted one. Without it, the preserved digest is attempted only when `preserved_regions` is given: `None` otherwise,

`Some(Err)` for invalid regions or a file over the in-memory limit. A library built without the `semantic` feature reports `built without the semantic feature` for the walk.

In the OCaml package, the walk reads the file through its own descriptor before the Rust core reads it for hashing. The Rust stamp check covers the hashing read; a caller that cannot rule out writes between the two should compare stamps or sizes itself. In the Rust core, a file up to 1 GiB is walked and hashed from the same bytes in memory.

Cost

A tiffgraph walk, which reads only directories and selected values and is bounded by tiffgraph's limits, then one SHA-256 pass over the file from the bytes already in memory. About the cost of the whole-file SHA-256.

The identities side by side

	SHA-256	DNG raw blocks	Image data	Lightroom fingerprint	Metadata record	Preserved	Block marks
Input	every byte	encoding tags, block counts and bytes of each primary raw image	strips and tiles of every directory, RW2 raw span, RAF CFA, or JPEG frame, tables and scans	decoded samples, camera model, crops, opcode lists	exiftool-rs record without location members, with engine stamp	every byte outside the primary XMP packet, with size and layout	length and two-end mark of every large block
Equality proves	identical files	same encoded raw data	same stored image bytes	Classic would pair the files	same exposed metadata, same engine	only the XMP packet can differ	same blocks, wherever they sit
Does not prove	—	metadata, interpretation tags, decoded or rendered equality	metadata, decoded or rendered equality	byte or pixel equality	anything about image data or unexposed bytes	anything about the XMP packet	bytes in a block's middle; metadata
Applies to	every regular file	DNG, ≤ 1 GiB	TIFF-family raws, TIFFs, RAF and JPEGs that are not DNGs, ≤ 1 GiB	supported DNG strata	any file the engine reads	DNG accepted by a tiffgraph grammar, ≤ 1 GiB	classic TIFF family and JPEG with a block of 512 bytes or more
Hash	SHA-256	SHA-256	SHA-256	MD5 (Lightroom's)	SHA-256	SHA-256	SHA-256 over sampled SHA-256 marks
Cost	one read	parse + raw bytes	parse + image bytes	full raw decode	metadata parse	walk + one pass	walk + two ends per block
Feature	always	always	always	`lightroom` (default)	`metadata` (GPL)	always; the Rust walk needs `semantic` (default)	always

Which identity answers which question

Is this file in my backup?

Start with **SHA-256**. If the backup holds a file with the same SHA-256, that file is an exact copy, and nothing more needs to be said.

If it does not, ask why. For a camera raw that is not a DNG, a TIFF or a JPEG, compare **image data**: an equal digest means the backup holds the same stored picture, with different metadata, such as a shifted capture time. For a DNG, compare **DNG raw blocks**: an equal raw-block digest means the backup holds the same encoded photograph, but with different bytes somewhere else. That is not yet a backup of this file. Next compare **preserved** digests under the same profile. If they are equal, the only difference is the XMP packet, and the backup plus this file's packet restores this file exactly. If they differ too, something outside the packet changed, perhaps a refreshed preview or a field a converter rewrote, and the file needs its own copy, or a verified delta such as dng-compare makes.

A failed identity at any step means "not established". It never means "contained".

Was this only a metadata edit?

Compare the two versions:

- SHA-256 differs, **preserved** equal under the same profile: only the XMP packet changed.
- The **metadata** digests then differ, and `metadata::diff` names the fields: a rating, a keyword, a develop setting.
- **preserved** differs but **raw blocks** are equal: the raw data is the same, but something besides the packet changed. Look at the layout, the previews and the private fields before calling it metadata-only.
- **raw blocks** differ: the encoded raw data changed. It may be a re-encode of the same samples, which dng-compare can decide by decoding, or a different photograph.

Where is this file after an editor wrote into it?

Compare **block marks**. An editor that writes ratings, captions or develop settings into a file keeps its large blocks, so the edited file has the same block marks at any path and under any name. This works for DNGs, other TIFF-based raws and JPEGs alike. It is a sampled identity, so treat a match as a strong hint.

Can I get the file back as it was before the edit?

Yes, if its **skeleton** was kept before the edit: `skeleton::rebuild` writes the old file from the skeleton and the edited file, and the result is checked against the old SHA-256. phindex keeps a skeleton for every file it indexes and `phindex restore` does the rebuild; DNG Compare's archives use the same skeletons for the files they refer to. It does not work when the edit replaced a block, such as a regenerated preview.

Is this the same shot after a capture-time shift?

When the camera clock was wrong and the times were fixed afterwards, every whole-file SHA-256 changed. For DNGs, compare **raw blocks**; for other camera raws, TIFFs and JPEGs, compare **image data**. Equal values mean the stored image is the one the camera wrote, and the **metadata** diff shows the shifted time. phindex's `align` makes this comparison across whole archives, from their indexes.

Is this still the original my catalog recorded?

Compute the **Lightroom fingerprint** and compare it, with the file size, against the `AgLibraryFileDigest.digest` value stored for that original, read from a copy of the catalog. Equal means Lightroom Classic's matcher would accept the file. A file outside the supported strata gives no answer from phidentify; it does not give a "no".

SHA-256 cannot answer this question, because the catalog does not store one, and because Classic's own value deliberately survives metadata written into the file.

Did the shuttle copy arrive intact?

When files travel on a portable drive or across a network and come back, the question is about bytes, so the answer is **SHA-256**: record it before the copy, recompute it after, and compare. Nothing weaker will do, because a damaged byte inside a raw block is still damage even if the metadata looks the same.

If an application on the other side wrote metadata into the files on arrival, SHA-256 will differ. The **preserved** digest then tells you whether the image data and layout arrived intact and only the packet changed, and the **metadata** diff tells you what was written.

Can I delete this copy?

No identity alone answers this, because each one leaves something out. Equal **SHA-256** means the bytes are the same, so this copy is redundant only if the other copy is kept and checked. Equal **raw blocks** or **image data** mean the photograph is the same, but the wrapper, the previews, the XMP and the develop history may not be. Deleting on that evidence loses them.

The safe answer is an archive that can rebuild every byte of the copy: DNG Compare's `dngc run`, fed by phindex indexes, records each file as identical to its counterpart, as a verified delta against it, or as a full copy. `dngc verify --restore-sample N` then rebuilds a sample and checks it. Delete only after that passes, and keep the archive and the counterparts it refers to.

Testing

The tests use only synthetic files, built in memory or written to a temporary directory.

- The Rust tests (31 unit tests and 6 integration tests, with the `metadata` feature) cover the raw-block identity ignoring XMP and placement but following every raw byte; a non-DNG TIFF and a truncated raw block reported as errors, not matches; each identity reported separately by `identify`; the preserved digest reproducing Unison's formula byte by byte and ignoring region contents; image data ignoring a rewritten, longer metadata packet and moved data in a TIFF-based raw, APP segments, comments and trailing data in a JPEG, and metadata before the raw data of an RW2 and a RAF, but following every image byte, and refusing a DNG, a JPEG cut before EOI and a non-image file; and, for the Lightroom fingerprint, crop and opcode parsing, the opcode fingerprint stream, the tiled digest and the refusal of unsupported strata.
- The skeleton tests (7, in `tests/skeleton.rs`) build a small TIFF with an XMP packet, a strip and a large private tag, a JPEG and a sidecar. They check that a file is rebuilt byte for byte from its skeleton and an edited version whose longer packet moved the blocks, in both directions, and that the two have the same block marks; that a changed block gives other block marks and a rebuild that fails naming the block; that a change in the middle of a long block keeps the marks and is caught by the whole-file SHA-256; that a sidecar is its own skeleton and a larger file of an unknown format has none; that a record and its packed bytes give the skeleton back, and wrong bytes or blocks out of order are refused; and that a file on disk and the same bytes in memory are cut alike. The cuts were also compared with dng-compare's Python implementation on its 167 synthetic corpus files that have a skeleton, with no difference.
- The OCaml test, run in both switches, checks a non-DNG file (raw blocks and Lightroom not attempted, `dng_semantic` and `image_data` failed, `preserved` not attempted), a missing file reported as an error, a JPEG's block marks, its skeleton from the read and the same skeleton from its structures alone, and the pool returning forty results in input order with four workers.

```
make test
(cd rust && cargo test --release --features metadata && cargo clippy --release --
features metadata --all-targets)
cd ocaml && opam exec --switch default -- dune test --force
cd ocaml && opam exec --switch 5.2.0+ox -- dune test --force
```

Synthetic tests check the code against its definitions. They are not the qualification of the Lightroom fingerprint, which rests on the native comparison described in its chapter, and they are not a survey of real camera files.

Licences

The phidentify source is MIT. Builds link third-party code under other licences:

- The default `lightroom` feature links rawler 0.8.0, under LGPL-2.1.
- The default `semantic` feature links tiffgraph's Rust crate, under MIT.
- The optional `metadata` feature links exiftool-rs through dng-compare's `dng-exiftool-ffi`, under GPL-3.0-or-later. A binary built with it is covered by the GPL. dng-compare's `rust-scanner/exiftool-ffi/THIRD-PARTY.md` lists the notices.

A build without `lightroom` and `metadata` links only MIT and permissively licensed crates.

Reference

Item	Value
Version	0.3.0
Rust	1.99.0, edition 2024, `unsafe` forbidden in the core
OCaml	5.1 or later, stock OCaml (`Domain.spawn`) or OxCaml (`Domain.Safe.spawn`)
In-memory limit	1 GiB; larger files get SHA-256, the DNG semantic walk from disk and the metadata record
Raw-block domain	`PHIDENTIFY-DNG-ENCODED-RAW-BLOCKS-1\0`
Image-data domain	`PHIDENTIFY-IMAGE-DATA-1\0`
Image-data prefix	`image_data::PREFIX_LEN` = 16 bytes; `candidate` accepts `FF D8 FF`, `FUJIFILMCCD-RAW`, `II`, `MM`
Image-data reasons	`NOT_APPLICABLE` = `not a TIFF-family file or JPEG`; `DNG` = `DNG: the raw-block identity applies instead`
Preserved domain	`SDE/0-preserved\0`
Lightroom engine revision	`native-original-digest-v5`
Maximum XMP packet for the walk	1 MiB
C ABI flags	`DNG_BLOCKS` 1, `LIGHTROOM` 2, `METADATA` 4, `COMPACT_RECORD` 8, `IMAGE_DATA` 16
C ABI field order	sha256, raw blocks, Lightroom, metadata digest, preserved, metadata record, image data
Rust features	`lightroom` (default), `semantic` (default), `metadata`
Source	github.com/firstpair/phidentify
Sister projects	dng-compare, tiffgraph, Semantic Unison
Indexer	phindex